Architecture SOA et services web

Chapitre 1: Technologies XML
Schéma XML



Schéma XML

- Un Schéma XML (XML schema Definition XSD) est un langage basé sur XML qui permet la définition de schémas (structure + type de données) des documents XML et facilite la communication entre applications.

- XML Schema est une alternative (successeur) pour les DTDs.

DTD	XML schema
N'est pas une syntaxe XML	Basé sur la syntaxe XML
Difficile à étendre	Facilement extensible
Données textuelles non typées	Supporte les types de données
Ne permet pas de spécifier exactement le nombre d'occurrences d'un élément	Permet de spécifier exactement le nombre d'occurrences d'un élément
Ne supporte pas les espaces de noms	Supporte les espaces de noms

Schéma XML

- Un schéma d'un document définit:
  - les **éléments** possibles dans le document
  - les **attributs** associés à ces éléments
  - la **structure** du document
  - les **types** de données

- Le schéma est spécifié en XML:
  - pas de nouveau langage
  - balisage de déclaration
  - espace de nom spécifique **xsd** : ou **xs**

- Présente de nombreux avantages
  - structures de données avec types de données
  - extensibilité par **héritage**
  - analysable à partir d'un parseur XML standard

Schéma XML: exemple

Un simple document XML

```
<?xml version="1.0"?>
<note>
 <to>Tove</to>
 <from>Jani</from>
 <heading>Reminder</heading>
 <body>Don't forget me this weekend!</body>
</note>
```

Un schéma XML

```
<?xml version="1.0"?>
<xs:schema xmlns:xs=http://www.w3.org/2001/XMLSchema
elementFormDefault="qualified">
<xs:element name="note">
 <xs:complexType>
  <xs:sequence>
   <xs:element name="to" type="xs:string"/>
   <xs:element name="from" type="xs:string"/>
   <xs:element name="heading" type="xs:string"/>
   <xs:element name="body" type="xs:string"/>
  </xs:sequence>
 </xs:complexType>
</xs:element>
</xs:schema>
```

Schéma XML: association entre un document XML et XSD

Cas sans namespace:

- Au niveau du xml

```
<?xml version="1.0" encoding="UTF-8"?>
<personnes xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:noNamespaceSchemaLocation="personnes.xsd">
    <personne>
        <nom>Ali</nom>
    </personne>
</personnes>
```

- Au niveau du xsd:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           elementFormDefault="unqualified">
  <xs:element name="personnes">
      ......
```

xs ou xsd

URI du namespace officiel de XML Schema.

- Facultatif (par défaut),
- Les éléments enfants **ne sont pas dans le namespace**, seuls les éléments racines le sont.

Schéma XML: association entre un document XML et XSD

Cas avec namespace:

- Au niveau du xml

namespace

```
<?xml version="1.0" encoding="UTF-8"?>
<ns:personnes xmlns:ns="http://www.exemple.com/personne"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:schemaLocation="http://www.exemple.com/personne personnes.xsd">

    <ns:personne>
        ........
```

Pour valider tout ce qui est dans le namespace, utilise….

- Au niveau du xsd:

Les éléments du XML appartiennent à ce namespace.

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.exemple.com/personne"
           elementFormDefault="qualified"
           xmlns:ns="http://www.exemple.com/personne">
    .....
```

- Tous les éléments définis dans le XSD **sont associés au namespace**.
- Tous les éléments enfants (ici personne, nom, age) appartiennent au namespace.

Schéma XML: commandes de base

- **Element**: association d'un type à une balise
  - name, type, ref, minOccurs , maxOccurs ,
- **Attribute** : association d'un type à un attribut
  - name , type
- **Type simple** : les multiples types de base
  - entier, réel, string, time, date, ID, IDREF, …,
  - extensibles par des contraintes
- **Type complexe** : une composition de types
  - définit une agrégation d'éléments typés

Schéma XML: Types prédéfinis

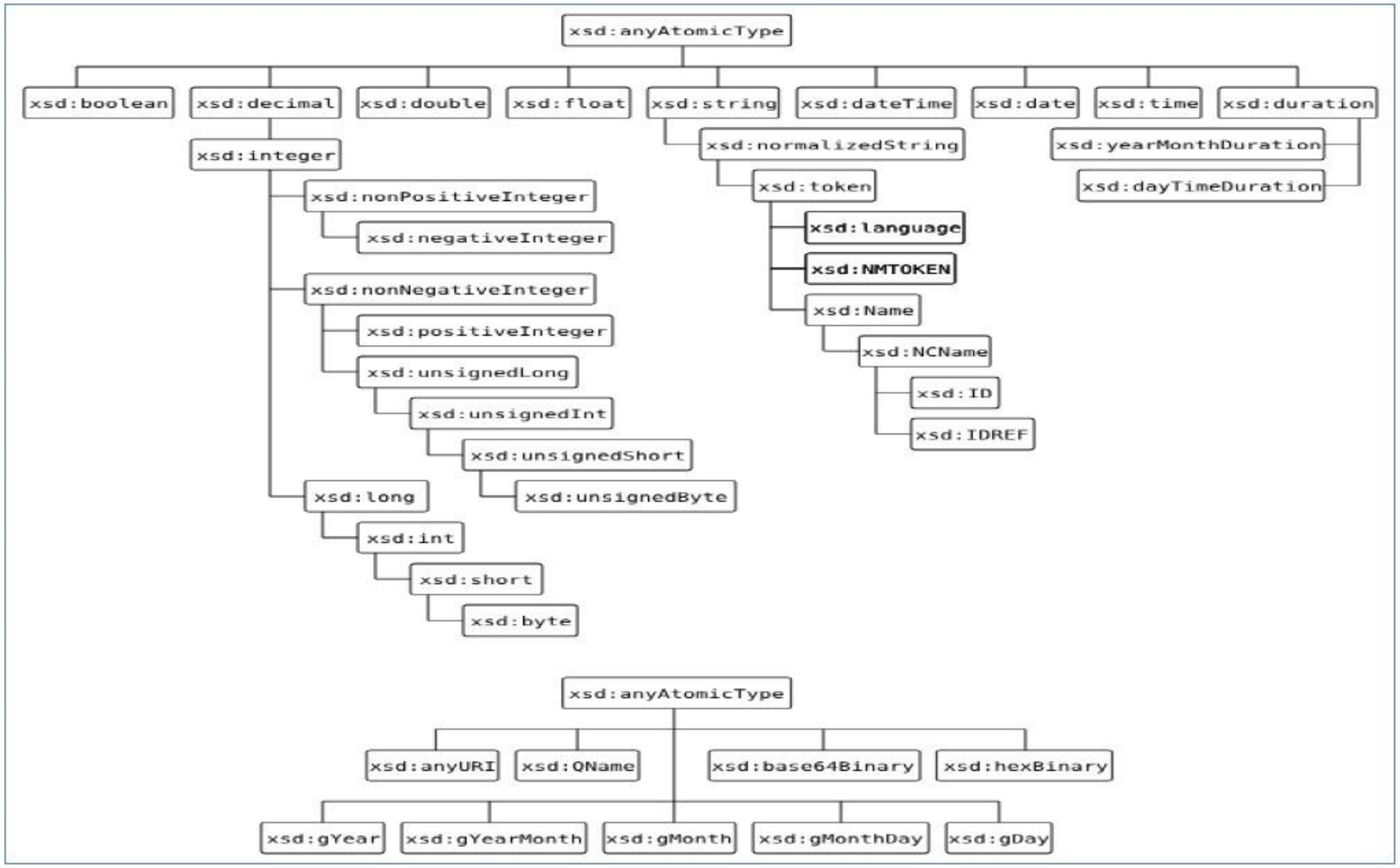


Schéma XML: Principaux types simples

- string
- byte :-128 et 127
- hexBinary : 0FB7
- integer :-126789, -1, 0, 1, 126789
- int : -2 147 483 648 et 2 147 483 647
- long : -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807.
- short :-32 768 et 32 767
- float : 3.4x10^-38 et 3.4x10^38
- double : 1.7x10^-308 et 1.7x10^308
- boolean : true, false 1, 0
- anyURI : http://www.xml.com

- time : 13:20:00
- dateTime : 2020-05-31T13:20:00
- date : 2023-05-31
- gMonth : --05--
- gYear : 2030
- gYearMonth :2026-02
- gDay :---31
- gMonthDay :--05-31
- language :en-GB, en-US, fr
- ID :"A212"
- IDREF: "A212"
- IDREFS :"A212" "B213"

Schéma XML: Principaux types simples

float

```
<xsd:element name="nombre" type="xsd:float" />
```

```
<nombre>42</nombre>
<nombre>-42.25</nombre>
<nombre>3E4</nombre>
<nombre>10e-5</nombre>
```

anyURI

```
<xsd:attribute name="adresse" type="xsd:anyURI"/>
```

```
<!-- URI absolu -->
<image adresse="http://www.xml.com/emplois/emploi2030.pdf" />
<!-- URI relatif -->
<image adresse="../emplois/emploi2030.pdf"/>
```

Schéma XML: Principaux types simples

ID et IDREF

```
<customer name="name 1" number="CUST12" type="VIP"/>
<orders count="2">
<order number="ORD100" owner="CUST12" total="900"/>
<order number="ORD101" owner="CUST12" total="200"/>
</orders>
```

```
<xs:element name="order">
<xs:complexType>
<xs:sequence>
<xs:attribute name="number" type="xs:ID" />
<xs:attribute name="owner" type="xs:IDREF" />
<xs:attribute name="total" type="xs:decimal" />
</xs:sequence>
</xs:complexType>
</xs:element>
```

Schéma XML: élément simple

- Un élément simple est un élément XML qui peut contenir du texte seulement. Il **ne peut pas contenir** d'autres éléments ou d'attributs.

```
<?xml version="1.0" encoding="UTF-8"?>
<age>25</age>
```

Exemple 1 : sans restriction

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001//XMLSchema">
<xsd: element name="age" type="xsd:int" />
</xsd:schema>
```

Schéma XML: élément simple

- Un élément simple est un élément XML qui peut contenir du texte seulement. Il **ne peut pas contenir** d'autres éléments ou d'attributs.

```
<?xml version="1.0" encoding="UTF-8"?>
<age>25</age>
```

Exemple 2 : extensible par des contraintes (l'âge doit être entre 15 et 29)

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001//XMLSchema">
<xs: element name="age">
 <xs:simpleType>
    <xs:restriction base="xs:int">
       <xs:minInclusive value="15"/>
       <xs:maxInclusive value="29"/>
    </xs:restriction>
 </xs:simpleType>
</xs:element>
```

Schéma XML: élément simple

1ère solution

```xml
<?xml version="1.0" encoding="UTF-8"?>
<voiture>Audi</voiture>
<voiture>Golf</voiture>
<voiture>BMW</voiture>
```

```xml
<xs:element name="voiture">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:enumeration value="Audi"/>
   <xs:enumeration value="Golf"/>
   <xs:enumeration value="BMW"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

2ème solution

```xml
<xs:element name ="voiture" type="Typevoiture"/>
<xs:simpleType name="Typevoiture">
 <xs:restriction base="xs:string">
  <xs:enumeration value="Audi"/>
  <xs:enumeration value="Golf"/>
  <xs:enumeration value="BMW"/>
 </xs:restriction>
</xs:simpleType>
```

```xml
<voiture>Mercedes</voiture> <!-- ✗ pas dans l'énumération -->
```

Schéma XML: élément simple

Valeur par défaut et fixe pour les éléments

Valeur par défaut :

```
<xs:element name = "grade" type = "xs:string" default = "Pr" />
```

Valeur fixe :

```
<xs:element name = "devise" type = "xs:string" fixed = "DT" />
```

```
<?xml version="1.0" encoding="UTF-8"?>
<grade/>
```

ou

```
<grade>Pr</grade>
```

→Si on ne fournit pas de valeur pour grade, la valeur par défaut Pr est utilisée.

→ Pour un élément à **valeur fixe**, le XML **doit obligatoirement** contenir cette valeur

```
<?xml version="1.0" encoding="UTF-8"?>
<devise>DT</devise>
```

Schéma XML: attribut

- Tous les attributs sont déclarés comme des types simples.
- Si un élément a des attributs, il est considéré comme étant de type complexe. Mais l'attribut lui-même est toujours déclaré comme un type simple.

Exemple

Voici un élément XML avec un attribut:

```
<lastname lang="EN">Smith</lastname>
```

Et voici la définition de l'attribut correspondant:

```
<xs:attribute name="lang" type="xs:string"/>
```

Valeur par défaut et fixe pour les attributs

Valeur par défaut :

```
<xs:attribut name = "grade" type = "xs:string" default = "Pr" />
```

Valeur fixe :

```
<xs:attribut name = "devise" type = "xs:string" fixed = "DT" />
```

Restriction :

```
 <xs:attribut name = "code" type = "xs:int" use = "required" />
```

Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

Restrictions sur les valeurs

```
<xs:element name="age">
 <xs:simpleType>
  <xs:restriction base="xs:integer">
   <xs:minInclusive value="0"/>
   <xs:maxInclusive value="120"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

Restrictions sur un ensemble de valeurs

```
<xs:element name="car">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:enumeration value="Audi"/>
   <xs:enumeration value="Golf"/>
   <xs:enumeration value="BMW"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

Restrictions sur une série de valeurs

```
<xs:element name="letter">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:pattern value="[a-z]"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

Expressions possibles

- `<xs:pattern value="[A-Z][A-Z][A-Z]"/>`
- `<xs:pattern value="[a-zA-Z][a-zA-Z][a-zA-Z]"/>`
- `<xs:pattern value="[xyz]"/>`
- `<xs:pattern value="[0-9][0-9][0-9][0-9][0-9]"/>`
- `<xs:pattern value="([a-z])*"/>`
- `<xs:pattern value="([a-z][A-Z])+"/>`
- `<xs:pattern value="m|f"/>`

Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

- Restriction sur la longueur

```
<xs:element name="password">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:length value="8"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

```
<xs:element name="password">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:minLength value="5"/>
   <xs:maxLength value="8"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

Schéma XML: élément complexe

- Un élément complexe est un élément XML qui contient d'autres éléments et / ou des attributs. Il existe quatre types d'éléments complexes:

  - Eléments vides

  - Contenu standard : il est composé d'éléments ou d'attributs
    - sequence : collection ordonnée d'éléments typés
    - all : collection non ordonnée d'éléments typés
    - choice : choix entre éléments typés

  - Contenu simple : il est composé d'attributs et d'un texte de type simple <xsd:simpleContent>

  - Contenu mixte : il est composé d'attributs, d'éléments et de texte

Schéma XML: élément complexe

- **Les éléments vides:**

Définition 1:seul l'élément product peut utiliser le type complexe spécifié

```
<!-- Définition de l'élément racine -->
  <xs:element name="product">
    <xs:complexType>
      <xs:attribute name="prodid"
type="xs:integer"/>
    </xs:complexType>
  </xs:element>
```

```
<product prodid="1345" />
```

Définition 2: plusieurs éléments peuvent se référer au même type complexe:

```
 <!-- Déclaration de l'élément racine
et association au type -->
  <xs:element name="product"
type="ProductType"/>
  <!-- Définition du type utilisé par
product -->
  <xs:complexType name="ProductType">
    <xs:attribute name="prodid"
type="xs:integer" />
  </xs:complexType>
```

Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent des attributs :

```
<personne nom= "Mabrouk" prenom="Mouna"/>
```

```
<personne nom="Mabrouk"/>
```

```
<xsd:element name="personne">
   <xsd:complexType>
     <xsd:attribute name="nom" type="xsd:string" use="required"/>
     <xsd:attribute name="prenom" type="xsd:string"/>
   </xsd:complexType>
 </xsd:element>
```

Obligatoire

Optionnelle

Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - Sequence

```
<personne>
  <nom>Mabrouk</nom>
  <prenom>Mouna</prenom>
</personne>
```

```
<xsd:element name="personne">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="prenom" type="xsd:string"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

- Ordre **obligatoire** : les éléments doivent apparaître dans l'ordre défini.
- Multiplicité contrôlé (avec minOccurs, maxOccurs).

Schéma XML: élément complexe

- **Contenu standard:** Indicateurs d'occurrence : Il est possible de contrôler le nombre minimal et maximal d'un élément par l'utilisation de **minOccurs** et **maxOccurs**

```
<personne>
  <nom>Mabrouk</nom>
  <prenom>Mouna</prenom>
  <email>mouna.Mabrouk@example.com</email>
  <email>contact.mouna@example.com</email>
</personne>
```

```
<xsd:element name="personne">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="prenom" type="xsd:string"/>
        <xsd:element name="email" type="xsd:string" minOccurs="1"
maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

un nombre illimité de fois

Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - all

```
<personne>
  <prenom>Mouna</prenom>
  <email>mouna.mabrouk@example.com</email>
  <nom>mabrouk</nom>
</personne>
```

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:all>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
      <xsd:element name="email" type="xsd:string" minOccurs="1"/>
    </xsd:all>
  </xsd:complexType>
</xsd:element>
```

- **Tous** les éléments doivent apparaître une fois (ou zéro fois si minOccurs="0"), mais **l'ordre n'a pas d'importance**.
- Chaque élément peut apparaître **au maximum** une fois (maxOccurs doit être 1)

Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - Choice

```
<personne>
  <nom>Mabrouk</nom>
</personne>
```

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
      <xsd:element name="email" type="xsd:string"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

- L'élément peut contenir un **seul** des sous éléments définis.
- Permet des alternatives.

Schéma XML: élément complexe

- **Contenu simple:** Les éléments qui contiennent du texte et des attributs:

```
<ville pays="Tunisie" population="200000">Tunis</ville>
```

```
<xsd:element name="ville">
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base="xsd:string">
        <xsd:attribute name="pays" type="xsd:string" use="required"/>
        <xsd:attribute name="population" type="xsd:integer"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:element>
```

Schéma XML: élément complexe

- **Contenu mixte:** Les éléments qui contiennent du texte et d'autres éléments :

```
<paragraphe>
  Bonjour, ceci est un
  <mot>exemple</mot>
  de contenu mixte
  <mot>XML</mot>.
</paragraphe>
```

```
<xsd:element name="paragraphe">
    <xsd:complexType mixed="true">
      <xsd:sequence>
        <xsd:element name="mot" type="xsd:string" minOccurs="0"
maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

Schéma XML: élément complexe

- **Contenu mixte:** Les éléments qui contiennent du texte, d'autres éléments et des attributs

```
<paragraphe langue="fr" id="1">
  Bonjour, ceci est un
  <mot>exemple</mot>
  de contenu mixte
  <mot>XML</mot>.
</paragraphe>
```

```
<xsd:element name="paragraphe">
  <xsd:complexType mixed="true">
    <xsd:sequence>
      <xsd:element name="mot" type="xsd:string" minOccurs="0"
maxOccurs="unbounded"/>
    </xsd:sequence>
    <xsd:attribute name="langue" type="xsd:string"/>
    <xsd:attribute name="id" type="xsd:integer"/>
  </xsd:complexType>
</xsd:element>
```

Schéma XML: élément complexe

- **Réutilisation des éléments**: On déclare de manière globale les différents éléments qui composent notre Schéma XML, puis, dans nos structures complexes, on y fait référence par le mot clef « **ref** ».

```
<xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="to"/>
        <xs:element ref="from"/>
        <xs:element ref="heading"/>
        <xs:element ref="body"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="to" type="xs:string"/>
  <xs:element name="from" type="xs:string"/>
  <xs:element name="heading" type="xs:string"/>
  <xs:element name="body" type="xs:string"/>
```

```
<note>
  <to>Alice</to>
  <from>Bob</from>
  <heading>Rappel</heading>
  <body>N'oublie pas notre
réunion demain à 10h.</body>
</note>
```

Schéma XML: élément complexe

- **Exercice :** Donner un schéma xml pour le fichier " shiporder.xml " suivant:

```
?xml version="1.0" encoding="ISO-8859-1"?>
<shiporder orderid="889923">
  <orderperson>John Smith</orderperson>
  <shipto>
    <name>Ola Nordmann</name>
    <address>Langgt 23</address>
    <city>4000 Stavanger</city>
    <country>Norway</country>
  </shipto>
  <item>
    <title>Empire Burlesque</title>
    <note>Special Edition</note>
    <quantity>1</quantity>
    <price>10.90</price>
  </item>
  <item>
    <title>Hide your heart</title>
    <quantity>1</quantity>
    <price>9.90</price>
  </item>
</shiporder>
```